

Windows uses drive letters (C:, D:).

Linux uses a single root (/).

Windows Folder Structure

```
C:\
├─ Program Files
├─ Users
│   ├─ Admin
│   └─ Student
├─ Windows
└─ Temp
```

- C:\ = main drive
- Everything starts from drives like C:\, D:\
- User files stored in: C:\Users\Student

Linux Folder Structure

```
/
├─ bin
├─ home
│   ├─ admin
│   └─ student
├─ etc
├─ usr
├─ var
└─ tmp
```

- / = root directory
- No C:, D:, or drives
- Everything starts from /

Program installation folders

In windows

C:\Program Files\

C:\Program Files (x86)\

In Linux

/bin

/usr/bin

/usr/local/bin

Example:

MySQL Installation Folder in Windows

```
C:\Program Files\MySQL\  
|  
├─ MySQL Server 8.0\  
|   └─ bin\  
|       └─ mysqld.exe  
|   └─ data\  
|   └─ docs\  
|   └─ include\
```

Everything is inside **one main folder**

C:\Program Files\MySQL\MySQL Server 8.0\

MySQL Installation Folder in Linux

Linux does not install MySQL in a single folder.

It spreads MySQL files according to their purpose.

/usr/bin/ → mysql client commands

/usr/sbin/mysqld → mysqld server executable file

/var/lib/mysql/ → database files

/etc/mysql/ → configuration files

/usr/share/mysql/ → SQL scripts & error messages

/var/log/mysql/ → logs

Where do we write Linux commands?

Linux commands are written in a program called the *Terminal*.

The terminal is like the **Command Prompt** or **Powershell** in Windows.

On Linux, it can be opened by:

pressing **Ctrl + Alt + T**, or clicking **Terminal icon**

Windows : PS C:\Users\DELL>

Linux: dell@linux:~\$

In windows, if you open Powershell, it will look like below

PS C:\Users\>\

In windows, if you open Command Prompt, it will look like below

C:\Users\>\

Example:

PS C:\Users\DELL>

Here,

PS → PowerShell

C: → Drive

\Users\DELL → Current folder

> → Prompt symbol

DELL → **username** of the logged-in user.

In Linux,

dell@linux:~\$ or sometimes: dell@ubuntu: ~\$

Here,

dell → username

@linux → Computer / Hostname

~ → Current folder or working directory

\$ → Prompt symbol

dell@linux:~\$ means dell@linux:/home/dell\$

pwd means: print working directory

It tells you the folder in which you are currently working.

dell@linux:~\$ pwd

/home/dell

If you move to Documents:

dell@linux:~\$ cd Documents

dell@linux:~/Documents\$ pwd

/home/dell/Documents

pwd does not work in Command Prompt of Windows

```
C:\Users\DELL>pwd
'pwd' is not recognized as an internal or external command,
operable program or batch file.
```

But, in **Windows Command Prompt (cmd.exe)**, the command to show the **current working directory** is : cd

```
C:\Users\DELL>cd
C:\Users\DELL
```

pwd works in **PowerShell**

```
PS C:\Users\DELL> pwd
Path
----
C:\Users\DELL
```

Command Prompt (Old Tool)	PowerShell (Modern & powerful)
Commands like ipconfig,cd	advanced than PowerShell .

when professionals use Command Prompt and when they use PowerShell?

When do professionals use **Command Prompt**?

Simple real example: Fixing an employee's internet problem

In companies, IT Support gets calls like: "Sir, my internet is not working."

IT person quickly opens **Command Prompt** and runs:

- ping google.com
- ipconfig
- tracert google.com

These commands check:

- Is internet working?
- Is the computer getting IP address?
- Where is the connection failing?

When do professionals use **PowerShell**?

Real example: Company has 300 computers → need to install software

Suppose a company wants to install **Google Chrome** on 300 computers at once.

Using mouse? Impossible. Takes days.

Using CMD? Cannot do bulk automation.

Using PowerShell? Yes

1st = Admin Person use **a file** or **Active Directory** to get all computer names automatically
for this, some command is used in powershell
This shows:

```
PC001
PC002
PC003
...
PC300
```

2nd = Export these names in one file e.g. `pcs.txt` file
This `pcs.txt` file is then used by scripts

3rd = In Powershell

```
$computers = Get-Content pcs.txt
Invoke-Command -ComputerName $computers `
-ScriptBlock {
    winget install Google.Chrome -e
}
```

What PowerShell does:

- Connects to each PC one by one
- Installs software
- Shows success or failure
- Admin just **presses Enter** and waits

Why do we need commands like *pwd* and *cd*, when we can simply see the folder path in Windows Explorer?

1. Command-line is MUCH faster than GUI

Imagine you want to go to: `C:\Users\DELL\Documents\Work\2024\Projects\WebApp\Source\Code`

Using GUI:

- Click folder
- Open sub-folder
- Open next
- ... 10–12 clicks!

Using command: `cd`

```
C:\Users\DELL\Documents\Work\2024\Projects\WebApp\Source\Code
```

- ➡ Professionals prefer speed.
- ➡ Developers & admins save **hours** using commands.

2. Because not all systems have a GUI

Many servers in the world run **Linux without any desktop** (no icons, no folders, no mouse).

Examples:

Web servers, Database servers, Cloud servers (AWS, Azure), Firewalls

These machines show **only a terminal**:

```
root@server:/var/www$
```

➡ For such systems, **pwd** and **cd** are the **only way to move around**.

cd - Change Directory

it is used to navigate between directories in the file system.

```
dell@linux:~$ ls
```

```
Output: Documents Downloads Music Pictures Videos test.txt
```

These are the folders and files inside the current directory.

```
dell@linux:~$ cd Downloads
```

Terminal becomes:

```
dell@linux:~/Downloads$
```

```
dell@linux:~/Downloads$ pwd
```

```
/home/dell/Downloads
```

Suppose you want to go to another folder (Documents)

Right now you are in Downloads:

```
dell@linux:~/Downloads$
```

But you want to go to **Documents**, which is one level above.

So first go **back**:

```
dell@linux:~/Downloads$ cd ..
```

Now terminal shows:

```
dell@linux:~$
```

```
dell@linux:~$ cd Documents
```

Now terminal:

```
dell@linux:~/Documents$
```

ls — list files

```
dell@linux:~$ ls
```

```
Documents Downloads Music Pictures Videos test.txt
```

mkdir — create folder

```
dell@linux:~$ mkdir College
```

```
dell@linux:~$ ls
```

```
College Documents Downloads Music Pictures Videos test.txt
```

rmdir — remove **empty** folder

```
dell@linux:~$ rmdir College
```

```
dell@linux:~$ ls Documents Downloads Music Pictures Videos test.txt
```

cat — view file content

(Assume test.txt already exists)

```
dell@linux:~$ cat test.txt
```

```
Hello Students
```

```
Welcome to Linux class
```

```
This is a sample file
```

cp — copy file

Copy *test.txt* to *Documents* folder:

```
dell@linux:~$ cp test.txt Documents/
```

```
dell@linux:~$ ls Documents
```

```
dell@linux:~$ Documents/test.txt
```

rm — remove file permanently

```
dell@linux:~$ rm test.txt
```

```
dell@linux:~$ ls
```

```
Documents Downloads Music Pictures Videos
```

mv — move OR rename file

Move file: dell@linux:~\$ mv Documents/test.txt Downloads/

Rename file: dell@linux:~\$ mv Downloads/test.txt old_test.txt

wc — count lines, words, characters

```
dell@linux:~$ wc old_test.txt
```

```
3 7 40
```

split — split big file

```
dell@linux:~$ split bigfile.txt
```

```
dell@linux:~$ ls
```

```
xaa xab xac Documents Downloads Music Pictures Videos
```

cmp — compare two files exactly (binary)

```
dell@linux:~$ cmp file1.txt file2.txt
```

```
file1.txt file2.txt differ: byte 12, line 2
```

If same:

```
dell@linux:~$ cmp file1.txt file1.txt
(no output → files are identical)
```

comm — compare two sorted files

```
dell@linux:~$ comm names1.txt names2.txt
```

Output example:

```
Amit
    Raj
        Zara
```

(Shows lines unique to file1, unique to file2, and common)

diff — show line-by-line difference

file1.txt (original):

```
1 Hello Students
2 Welcome to Linux class
3 This is a sample file
4 End of file
```

file2.txt (modified):

```
1 Hello Students
2 Welcome to Advanced Linux class
3 This is a sample file
4 New line here
5 End of file
```

```
dell@linux:~$ diff file1.txt file2.txt
```

```
2c2
< Welcome to Linux class
---
> Welcome to Advanced Linux class
3a4
> New line here
```

2c2 → **change**: line 2 in file1.txt was changed into line 2 in file2.txt.

The lines **starting with <** are from **file1** (old).
The lines **starting with >** are from **file2** (new).

3a4 → **add**: after line 3 in file1.txt, file2.txt has a new line (which is line 4 in file2.txt).

> New line here is the added line (present only in file2).

Symbols quick reference:

NcM : line N in file1 **changed** to line M in file2.

NdM : line N in file1 **deleted** (doesn't appear in file2).

NaM : after line N in file1, lines from file2 starting at M were **added**.

< : shows content from first file.

> : shows content from second file.

head — show first lines

```
dell@linux:~$ head old_test.txt
Hello Students
Welcome to Linux class
This is a sample file
```

Linux shows the **first 10 lines** of the file. Because the **default is 10 lines**. Even if your file has only 3 lines, it will show all 3.

First 2 lines:

```
dell@linux:~$ head -2 old_test.txt
Hello Students
Welcome to Linux class
```

tail — show last lines of a file

Let's assume our file: **notes.txt** contains:

```
Welcome to Linux
We are learning commands. tail shows last lines.
grep is used to search
Line 4: sort arranges data
Thank you students
```

```
dell@linux:~$ tail notes.txt
Welcome to Linux
We are learning commands. tail shows last lines.
grep is used to search
Line 4: sort arranges data
Thank you students
```

Because the file has only 5 lines.

```
dell@linux:~$ tail -2 notes.txt
Line 4:sort arranges data
Thank you students
```

grep — search text inside file

Search the word "Linux":

```
dell@linux:~$ grep "Linux" notes.txt
Welcome to Linux
```

```
dell@linux:~$ grep "line" notes.txt  
tail shows last lines
```

```
dell@linux:~$ grep "Line" notes.txt  
tail shows last lines  
Line 4:sort arranges data
```

sort — alphabetically sort file lines

arrange file content alphabetically

Let's assume a file **names.txt** contains:

Zara
Rahul
Amit
Pooja
Jatin

```
dell@linux:~$ sort names.txt
```

Output:

Amit
Jatin
Pooja
Rahul
Zara

```
dell@linux:~$ sort -r names.txt
```

save sorted output to new file

```
dell@linux:~$ sort names.txt > sorted_names.txt
```

Check file:

```
dell@linux:~$ cat sorted_names.txt
```

sorting numbers :

```
dell@linux:~$ sort -n numbers.txt
```